Uppsala University



Machine Learning

Assignment 8

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution.
- State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available in **the PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

```
1. install.packages("remotes")
```

2. `remotes::install_github("MansMeg/IntroML", subdir = "rpackage")`

- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You are not allowed to show your assignments (text or code) to anyone. Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. Similarly, on zoom labs, only screen share when you are in a separate zoom room with teaching assistants.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your submitted work orally.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Do not copy or upload the full assignment text into an LLM.
- The computer labs are for asking all types of questions. Do not hesitate to ask! The purpose of the computer labs are to improve your learning. We will hence focus on more computer labs and less on assignment feedback. *Warning!* There might be bugs in the assignments! Hence, it is important to ask questions early on so you dont waste time of unintentional bugs.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Contents

Oral exam question contribution 4

1 Bandits 5

2 Markov Decision Processes 9

3 * Some additional bandit algorithms 10

Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 2–4 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

1 Bandits

In the first part, we will study the k-armed bandit problems more in detail.

1. (2p) Implement a stationary 5-armed bandit reward function that generates a reward R for a given category according to a true underlying (stationary) reward. Implement a function `stationary_bandit(a)` that takes an action $\mathcal{A} = (1, ..., 5)$. The reward function should return a reward according to

$$R \sim N(q_a^\star, 1)\,,$$

where $q^\star = (1.62, 1.20, 0.70, 0.72, 2.03)$. Below is an example of how it *could* work. Note that since the function returns a random value, you might get a different result, and it might still be a correct implementation. However, to test, you can run your bandit 1000 times and take the mean (see below). Then you should be close to $q^\star$ if your implementation is correct.

```
set.seed(4711)
stationary_bandit(2)

## [1] 3.019735

stationary_bandit(1)

## [1] 2.99044

R <- numeric(1000)
for(i in 1:1000){
  R[i] <- stationary_bandit(3)
}
mean(R)

## [1] 0.6819202
```

2. (2p) Also, we will also implement a non-stationary reward function. For all actions $\mathcal{A} = (2, ..., 5)$ the non-stationary should work as the stationary reward function. Although for action $a = 1$, the function should return a reward based on the iteration according to

$$R \sim N(-3 + 0.01t, 1)\,.$$

Implement the reward function as `nonstationary_bandit(a, t)` and it should work as follows. Again, note that these are random values, you might have another (correct) solution giving different results.

```
set.seed(4711)
nonstationary_bandit(2, t = 1)

## [1] 3.019735
```

```r
nonstationary_bandit(2, t = 1000)
```

```
## [1] 2.57044
```

```r
nonstationary_bandit(1, t = 1)
```

```
## [1] -1.793682
```

```r
nonstationary_bandit(1, t = 1000)
```

```
## [1] 6.593121
```

3. (2p) Implement a greedy policy algorithm that always chooses the greedy action, according to Eq. (2.2) in Sutton and Barto (2020). The function called `greedy_algorithm(Q1, bandit)` should input a vector of initial estimates Q_1 and a reward function. Then the algorithm should run the bandit for 1000 steps and return the following; (1) The rewards R_t for all steps, (2) the mean reward over all the 1000 steps, (3) the value estimates $Q_t(a)$, and (4) the total number of choices of each action $N_t(a)$. Here is an example of how it could work. *Note!* This is a random algorithm so you might get a different result, even if you use the exact same seed and have a correct algorithm.

```r
Q1 <- rep(0, 5)
set.seed(4711)
greedy_algorithm(Q1, stationary_bandit)
## $Qt
## [1] 1.602744 0.000000 0.000000 0.000000 0.000000
##
## $Nt
## [1] 1000    0    0    0    0
##
## $R_bar
## [1] 1.602744
##
## $Rt
## [1]  3.439735114  2.990439504  2.816318235  1.213120797 ...

set.seed(4711)
Q1 <- rep(0, 5)
greedy_algorithm(Q1, nonstationary_bandit)
## $Qt
## [1] -1.170265  1.180905  0.000000  0.000000  0.000000
##
## $Nt
## [1] 1 999   0   0   0
##
## $R_bar
## [1] 1.178554
##
## $Rt
## [1]  -1.170264886  2.570439504  2.396318235  0.793120797 ...
```

4. (2p) Similarly, implement the ϵ-bandit algorithm that takes an additional argument ϵ that is the probability that the algorithm instead takes makes an explorative action. Implement this algorithm as `epsilon_algorithm(Q1, bandit, epsilon)`, note that it should return the same output structure as the greedy algorithm. It is also clear that the randomness of the exploration can give quite some different results in different runs.

```
set.seed(4711)
Q1 <- rep(0, 5)
epsilon_algorithm(Q1, stationary_bandit, 0.1)
## $Qt
## [1] 1.6093183 1.1491153 0.7382012 0.5975228 2.0272784
##
## $Nt
## [1] 234  17  19  16 714
##
## $R_bar
## [1] 1.867178
##
## $Rt
## [1]  1.77251764  2.81631824  0.63350863  2.01754941 ...

set.seed(4711)
Q1 <- rep(0, 5)
epsilon_algorithm(Q1, nonstationary_bandit, 0.1)
## $Qt
## [1] 1.5887316 1.0956679 0.6962471 0.5975228 2.0468788
##
## $Nt
## [1]  24 159  20  16 781
##
## $R_bar
## [1] 1.834438
##
## $Rt
## [1]  -2.83748236  2.39631824  0.21350863  2.01754941 ...
```

5. (2p) Now implement the non-stationary bandit algorithm in a similar way with parameter α as `nonstationary_algorithm(Q1, bandit, epsilon, alpha)`.

```
set.seed(4711)
Q1 <- rep(0, 5)
nonstationary_algorithm(Q1, stationary_bandit, epsilon = 0.1, alpha = 0.2)
## $Qt
## [1] 1.414101 1.293336 1.237085 0.517214 2.134716
##
## $Nt
## [1] 302  16  19  16 647
##
## $R_bar
## [1] 1.840128
##
## $Rt
```

```
## [1] 1.77251764  2.81631824  0.63350863  2.01754941 ...

set.seed(4711)
Q1 <- rep(0, 5)
nonstationary_algorithm(Q1, nonstationary_bandit, epsilon =  0.1, alpha = 0.2)
## $Qt
## [1] 7.055805 1.287957 1.243996 0.517214 2.569759
##
## $Nt
## [1] 387 193  70  16 334
##
## $R_bar
## [1] 2.818588
##
## $Rt
## [1] -2.83748236  2.39631824  0.21350863  2.01754941 ...
```

6. (2p) Run 500 simulations of each algorithm and compute the mean of the mean rewards for each algorithm for both the stationary and non-stationary bandits. Do this for the greedy, the ϵ-algorithm, and the non-stationary algorithm. Try both $\alpha = 0.5$ and $\alpha = 0.9$ for the non-stationary algorithm. For both the non-stationary and the ϵ-algorithm, use $\epsilon = 0.1$. Summarize all the results in a table. You should have eight different results (four per bandit and two per algorithm).

   Note! The algorithm should run 1000 steps for each of the 500 simulations.

7. (2p) What are your conclusions from these results? Which algorithm seems to be the best one, and why?

8. (2p) Plot the average reward per step for the 500 runs for the worst and best algorithm, such as in Figure 2.2. in Sutton and Barto (2020). Use the output R_t from your implementations.

2 Markov Decision Processes

Now we are going to implement a Markov Decision Process (MDP) where the agent makes a decision, A_t based on the current state S_t and a reward R_t that will, in turn, return a new state S_{t+1} and a reward R_{t+1}. In this part of the assignment, we are going to use the recycling robot example (Example 3.3 in Sutton and Barto, 2020).

1. The MDP in Sutton and Barto (2020) has been implemented as a function `recycling_mdp(alpha, beta, r_wait, r_search)` in the `uuml` R package and can be accessed as follows.

```
library(uuml)
recycling_mdp(0.5, 0.8, 0.1, 1)

##       s s_next        a p_s_next r_s_a_s_next
## 1  high   high   search      0.5          1.0
## 2  high    low   search      0.5          1.0
## 3   low   high   search      0.2         -3.0
## 4   low    low   search      0.8          1.0
## 5  high   high     wait      1.0          0.1
## 6  high    low     wait      0.0          0.1
## 7   low   high     wait      0.0          0.1
## 8   low    low     wait      1.0          0.1
## 9   low   high recharge      1.0          0.0
## 10  low    low recharge      0.0          0.0
```

2. (2p) Now implement a function `always_search_policy(MDP)` that takes an MPD specified above and run the MDP for 1000 steps/actions. The policy should be that the agent always chooses to search, irrespective of the state. The function should return the return divided by the number of time steps and times in each state. The robot should start in the state `high`. Below is an example of how it should work. Note again, since this is a random algorithm, you might get a slightly different answer.

```
set.seed(4711)
always_search_policy(mdp)
## $Nt
## high low
## 288  712
##
## $R_bar
## [1] 0.452
```

3. (2p) Now implement a function `charge_when_low_policy(MDP)` that takes an MPD specified above and run the MDP for 1000 steps. The policy should be that the agent always chooses to recharge if the energy level is low and always search if the energy level is high. The function should return the same output structure as above.

4. (2p) Compare the two policies for MDP with $\alpha = \beta = 0.9$ and $\alpha = \beta = 0.4$ with $r_{\text{wait}} = 0.1$ and $r_{\text{search}} = 1$. Run at least ten times per policy and compute the mean reward. What policy is prefered in the two MDPs?

3 * Some additional bandit algorithms

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

We are here going to implement two additional bandits and compare them to the previous implementations above.

1. (2p) Implement the UCB algorithm as a function called `ucb_algorithm(Q1, bandit, c)`. *Hint!* If $N_t(a) = 0$, then a should be considered as the maximizing action (i.e. should be chosen). This can be implemented in slightly different ways, so your output might not match the output below exactly even if the implementation is correct. The example output is based on choosing each arm randomly as long as any $N_t(a) = 0$.

```
set.seed(4711)
Q1 <- rep(0, 5)
ucb_algorithm(Q1, stationary_bandit, c = 2)
## $Qt
## [1]  1.4792328 1.0928462 0.8417744 0.8360722 1.9867061
##
## $Nt
## [1]  64 26 16 17 877
##
## $R_bar
## [1] 1.893108
##
## $Rt
## [1]  1.3525176 1.9163182 0.6335086 ...

set.seed(4712)
Q1 <- rep(0, 5)
ucb_algorithm(Q1, stationary_bandit, c = 2)

## $Qt
## [1] 1.5993426 0.9645111 0.4399236 0.6868328 1.9736659
##
## $Nt
## [1]  98 21 10 14 857
##
## $R_bar
## [1] 1.882437
##
## $Rt
## [1]  -0.200366 1.848627 -0.2808972 ...
```

2. (2p) Implement the gradient bandit algorithm as a function called `gradient_bandit_algorithm(bandi alpha)`. Initialize all $H_t(a) = 1$. The algorithm should print the final preference and the probability of each action instead of Q_t.

```r
set.seed(4711)
gradient_bandit_algorithm(stationary_bandit, alpha = 0.1)
## $Ht
## [1] -0.4771226 -1.0045831 -1.7018038 -1.7427317  4.9262412
##
## $pit
## [1] 0.004 0.003 0.001 0.001 0.990
##
## $Nt
## [1]  27  24  11  12 926
##
## $R_bar
## [1] 1.941705
##
## $Rt
## [1]  1.772517635  2.396318235 -0.286491374 ...

set.seed(4712)
gradient_bandit_algorithm(stationary_bandit, alpha = 0.1)
## $Ht
## [1] -1.217231 -1.201571 -1.938602 -1.241492  5.598897
##
## $pit
## [1] 0.001 0.001 0.001 0.001 0.996
##
## $Nt
## [1]  26  30  18  20 906
##
## $R_bar
## [1] 1.885579
##
## $Rt
## [1]  0.62963402  0.94862700 -0.30089716 ...
```

3. (2p) Test to run the gradient bandit with $\alpha = 0$. What happens and why?

4. (2p) Run the gradient bandit and the UCB algorithms 500 times/simulations and compute the mean of the mean rewards (for each simulation) for each of the algorithms. *Note!* The algorithm should run 1000 steps for each of the 500 simulations. Use both the stationary and non-stationary bandit. For the UCB, try $c = 0.5$ and $c = 2$ and for the gradient bandit $\alpha = 0.1$ and $\alpha = 1$. Summarize all the results in a table and compare the results with the previous bandit algorithms. Which algorithm performs the best?

References

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction (Second edition)*. MIT press, 2020.